

Zuker Algorithm Implementation for RNA Secondary Structure Prediction

Motivation

Formulation: Given an RNA sequence $S \in \{A, U, G, C\}^*$, find the non-crossing secondary structure P with Minimal Free Energy (MFE).

Advantages over Nussinov Algorithm

- Minimizes free energy instead of maximizing the number of base pairs.
- Uses experimentally determined free energies rather than treating all base pairs equally.

Recurrence Relations

The optimal RNA secondary structure is computed using a 3D dynamic programming approach.

General Recurrence (W Matrix)

Let $W_{i,j}$ represent the MFE of the subsequence $S[i..j]$.

$$W_{i,j} = \min \begin{cases} W_{i,j-1} & S[j] \text{ unpaired} \\ \min_{i \leq k < j-1} W_{i,k-1} + V_{k,j} & \text{otherwise} \end{cases}$$

The final MFE is $W_{1,n}$.

Paired Recurrence (V Matrix)

Let $V_{i,j}$ represent the MFE of the subsequence $S[i..j]$ where bases i and j are paired.

$$V_{i,j} = \min \begin{cases} \text{hairpin}(i, j) \\ \text{stacking}(i, j) + V_{i+1,j-1} \\ \text{internal}(i, j) \\ \text{multiloop}(i, j) \end{cases}$$

- **Hairpin:** Energy for a hairpin loop of size $j - i - 1$.

- **Stacking:** Energy for stacking pair plus the MFE of the enclosed region.
- **Internal/Bulge:** Minimum energy over possible internal loops, considering asymmetry.
- **Multiloop:** Energy for multiloop structures.

Multi-loop Recurrence (WM Matrix)

Let $WM_{i,j}$ represent the MFE of a multi-loop region in $S[i..j]$.

$$WM_{i,j} = \min \begin{cases} WM_{i,j-1} + c & S[j] \text{ unpaired} \\ WM_{i+1,j} + c & S[i] \text{ unpaired} \\ V_{i,j} + b & \text{closed stem} \\ \min_{i < k < j} WM_{i,k} + WM_{k+1,j} & \text{split} \end{cases}$$

Where a, b, c are multiloop parameters (offset, helix penalty, unpaired nucleotide penalty).

Complexity

- **Time Complexity:** $O(n^3)$. The algorithm uses three DP tables of size $O(n^2)$, and filling each cell involves $O(n)$ operations for finding minimums in the V and WM matrices. By restricting the internal loop size to 30, the computation for internal loops is bounded to $O(1)$ per cell, maintaining the overall cubic complexity.
- **Space Complexity:** $O(n^2)$, due to storing the three DP tables (V, W, WM) of size $n \times n$.

Architecture

- `lookup.py` : Handles energy lookup tables for stacking, hairpin, bulge, internal, and multiloop energies. Parses data files and creates JSON caches.
- `stack.json` , `bulge.json` , `hairpin.json` , `internal.json` : JSON files containing precomputed energy values.
- `zucker.py` : Main implementation of the Zuker algorithm, including DP tables (V, W, WM) and backtracing for structure prediction.
- `validation.py` : Scripts for validating the implementation against known structures and datasets.

Validation

Validation is performed in two stages:

1. **Basic Tests:** Use short sample sequences to verify that stacking and backtracing work correctly. For these, stacking should be preferred over loops.
2. **Extended Tests:** Test with longer sequences and model tRNA structures.

Dataset: Archivell

- Source: [Hugging Face Archivell Dataset](#)
- Provides RNA sequences and corresponding dot-bracket notations for comparison.

Running Validation

Use `validation.py` to run validation tests. The script accepts the following inputs:

- `--input sample` : Run basic tests with short sample sequences.
- `--input long` : Test with longer sequences.
- `--input trna` : Validate against tRNA structures.
- `--input archivell` : Validate using the Archivell dataset.
- `--input test_string --sequence <seq> --canonical <dot_bracket>` : Test a custom RNA sequence with its canonical dot-bracket structure (canonical is required for this option).

Statistics computed:

- **F1 Score:** Harmonic mean of precision and recall.
- **Precision:** Fraction of predicted base pairs that are correct.
- **Recall:** Fraction of true base pairs that are predicted.
- **Tuple List:** A list of tuples indicating positions; +1 for missing or mismatched base pairs.

Example usage:

```
python validation/validation.py --input "sample"
python validation/validation.py --input "AUGC" --canonical "(.)"
```

Future Steps

- **Custom Lookup Tables:**
 - Hairpin: Factor in closing nucleotides.
 - Internal: Same as above.
 - Multiloop: Vectorize a , b , c to account for every inner loop.

Citations

- Algorithm: [Zuker Algorithm Slides](#)
- Dataset: [Archivell on Hugging Face](#)